```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files dataset.csv

dataset.csv(text/csv) - 600911 bytes, last modified: 8/3/2026 - 100% done

```
Saving dataset.csv to dataset.csv
```

```
df = pd.read_csv('dataset.csv')
print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

```
df = df.fillna(0)
```

```
cols = df.columns[1:]
df_flattened = pd.get_dummies(df.iloc[:, 1:], prefix='', prefix_sep='').groupby(level=0, axis=1).max()
df_flattened['Disease'] = df['Disease']

X = df_flattened.drop('Disease', axis=1)
y = df_flattened['Disease']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
/tmp/ipykernel_379/2935421736.py:2: FutureWarning: DataFrame.groupby with axis=1 is deprecated. Do `frame.T.groupby(...)` without axis instead.
  df_flattened = pd.get_dummies(df.iloc[:, 1:], prefix='', prefix_sep='').groupby(level=0, axis=1).max()
```

```
models = {
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    print(f"{name} Accuracy: {acc*100:.2f}%")
```

```
Decision Tree Accuracy: 100.00%
Random Forest Accuracy: 100.00%
KNN Accuracy: 100.00%
```

```
def predict_disease(symptoms_list):
     input_data = pd.DataFrame(0, index=[0], columns=X.columns)
     for s in symptoms_list:
        if s in input_data.columns:
            input_data[s] = 1

            rf_model = models["Random Forest"]
            prediction = rf_model.predict(input_data)[0]
            probabilities = rf_model.predict_proba(input_data)
            max_prob = np.max(probabilities)

            return prediction, max_prob
```

```
disease, prob = predict_disease(['itching', 'skin_rash', 'nodal_skin_eruptions'])
print(f"Predicted: {disease} (Confidence: {prob*100:.2f}%)")
```

```
Predicted: Fungal infection (Confidence: 32.00%)
```